

Personalised Multimodal Recipe Agent: A Component-Level Comparison from Plain LLM to LangGraph

Yutong Wu

School of EECS, The University of Queensland
Brisbane, Australia
s4779072@student.uq.edu.au

Abstract

We build a personalised recipe-QA testbed and use it to quantify the *marginal* contribution of each component in a typical multimodal agent stack. Four progressively more sophisticated systems (V0 plain LLM, V1 text RAG, V2 multimodal RAG, V3 LangGraph agent) and three V3 ablations (no-memory, no-planner, no-verifier) are evaluated over 20 queries spanning factual, cross-modal, multi-hop, and conversational families, with fully programmatic rule-based scoring against a 20-recipe knowledge base. Retrieval, multimodal fusion, memory, and verification are each shown to contribute positively. However, we identify a *negative* marginal benefit of structured planning on multi-hop and conversational queries: removing the planner from V3 raises overall task success from 65% to 75%, at $\sim 3.6\times$ the token cost. The result challenges the assumption that more architecture is monotonically better and supports a query-type-dependent design.

1 Introduction

Retrieval-Augmented Generation (RAG) and agent frameworks such as LangGraph have made it tractable to build multimodal QA systems that combine retrieval, planning, tool use, and memory. A common assumption is that each added layer of complexity yields a monotonic improvement in answer quality. However, the *component-level marginal benefit* of each layer is rarely measured directly: most published systems report a single end-to-end number against a baseline without isolating which component carries the gain.

We frame this work as a systems study rather than a system proposal. Specifically, we ask: *retrieval aside*, do *multimodal* indices, *persistent memory*, *explicit planning*, and *constraint verification* each contribute positive marginal value, and on which query types?

Contributions.

- A controlled comparison of 7 system configurations (4 progressive baselines plus 3 V3 ablations) on the same 20-query test set, with the underlying LLM held constant.
- A 20-recipe personalised knowledge base co-designed with the test queries to enable *purely rule-based* evaluation, with verifiable constraints replacing LLM-as-judge.
- A 5-class failure taxonomy aligned with the RAG Triad [3], separating retrieval, grounding, reasoning, query-grounding, and citation failures.
- A counter-intuitive finding: **structured planning has negative marginal benefit on multi-hop and conversational queries**, suggesting that explicit plans can be more brittle than reactive tool calling when query semantics are ambiguous.

2 Personalised Knowledge Base

The knowledge base contains 20 recipes (`data/recipes.json`) with paired AI-generated photographs (`images/`). Each recipe carries structured fields: cuisine, prep / cook / total minutes, servings, main protein, ingredients (name, amount, unit), dietary tags, allergens, cooking methods, step list, an image caption, and a *visual attributes* block (dominant colours, presentation, visible ingredients).

The recipe set is intentionally curated so that gold answers are computable from the structured fields, not opinion-based:

- Three recipes containing *eggs*, *tomato*, and *onion* (cook times 8 / 30 / 40 min), enabling a deterministic argmin test (QF3.1).
- Exactly two recipes containing both *garlic* and *ginger* (QF3.3 set-intersection test).
- Three Thai dishes, two with peanuts and one without (QF4.1 allergen test).
- Visually similar but cuisine-distinct distractor pairs (tonkatsu / tempura / fried_chicken; curry-udon / pad-thai) used in adversarial cross-modal probes (QF2.4, QF2.5).
- A vegan recipe with the shortest prep time (avocado-toast, 2 min; QF3.2 superlative test).

This adversarial co-design lets us check answers programmatically against hard facts, eliminating self-evaluation bias.

3 Retrieval Design

Both text and images are encoded into a shared joint space using *OpenCLIP ViT-B-32* [1] (LAION-2B pretrained weights). Two ChromaDB [5] collections are persisted under `chroma_db/: recipes_text`, where the document per recipe concatenates name, cuisine, ingredients, methods, dietary tags, and image caption; and `recipes_image`, where each document is the CLIP image embedding of the recipe photograph.

We use *two separate indices* rather than a single fused index deliberately: this lets the V3 planner choose between modalities (and lets us ablate that choice). V2 always queries both indices and merges candidates by interleaved rank, deliberately exercising the “always-fuse” opposite of V3’s routing.

4 Agent Workflow

V0–V2 baselines. V0 is a plain LLM call. V1 retrieves top-3 text candidates and passes them as context. V2 retrieves top-3 from both text and image indices, fuses them by interleaved rank, and passes the union (up to 5) as context. No memory, no tools.

V3 (full agent). A LangGraph [4] state machine with six nodes and five tools (Figure 1). The planner produces a structured JSON plan over the five tools; the executor runs each step with `$step_N`

reference resolution; the verifier re-extracts hard constraints from query + memory and filters candidates accordingly; the generator writes the final answer; `memory_update` extracts durable facts (allergies, dislikes, cooked-yesterday) from the user’s latest message and persists them.

Single-LLM principle. Every LLM call across V0–V3 (planner, verifier, generator, `memory_update`, V0/V1/V2 single-shot) uses the same DeepSeek V4 Flash [2] instance via the OpenAI-compatible API at temperature 0. This isolates architectural choices as the only variable between systems.

Ablations. Three V3 configurations disable one node group each: V3-NO-MEMORY skips `memory_load` and `memory_update`; V3-NO-PLANNER replaces the planner+executor with a reactive `bind_tools` loop where the LLM decides tool calls iteratively (capped at 6 rounds); V3-NO-VERIFIER passes executor results directly to the generator.

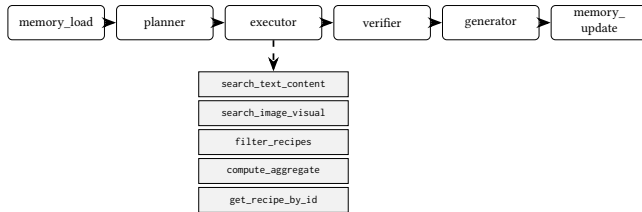


Figure 1: V3 agent. Six LangGraph nodes operate over a shared state; the executor dispatches to five callable tools. Ablations disable `memory_*` (no-memory), verifier (no-verifier), or replace planner+executor with reactive `bind_tools` (no-planner).

5 Experimental Setup

Query set. 20 queries are split equally across four families (5 each): **factual** (single-fact look-up, plus QF1.5 as an out-of-KB hallucination probe), **cross-modal** (visual property queries, including two adversarial cuisine-exclusion probes), **multi-hop** (filter-then-aggregate, set intersection, multi-constraint), and **conversational** (2–3 turn scripts testing allergen accumulation, negative recall, preference conflict, and constraint override).

Constraints. Each query carries one or more verifiable constraints drawn from 11 deterministic check types (*contains-number*, *mentions-recipe*, *does-not-mention*, *subset-of*, *recipe-count*, *indicates-no-recipe*, *identifies-superlative*, *acknowledges-conflict*, etc.). *Task success* is 1 iff all constraints for the query pass.

Metrics. We report Recall@3 against gold recipe IDs (skipped when the query is out-of-KB or aggregate-valued); task success; faithfulness (numeric- and name-claim support against retrieved context); visual grounding (cross-modal only; colour and presentation words checked against *visual attributes*); and three efficiency metrics (end-to-end latency, tool calls, and input+output tokens).

Memory contract. Memory is reset between test cases, not between turns within a multi-turn case, ensuring within-conversation context for V3 but no leakage between independent queries.

Caching and reliability. Each (system, query) record is cached after a fully successful run; failed records are logged and

never written to cache, so re-runs retry only failed pairs. The LLM wrapper retries 429/5xx errors with exponential backoff (max 3 attempts) and otherwise raises to the outer loop. All 140 records succeeded in the final run reported here.

6 Results

6.1 Overall task success

Table 1 and Figure 2 show task success per (system, family). The headline pattern is that *complexity is not monotonically beneficial*: while V0→V1→V2 shows progressive gains, the move from V2 to the full V3 agent *loses* 0.2 absolute points on multi-hop and conversational. The best overall system is in fact V3-no-planner (15/20 = 0.75), revealing that V3’s structured planner is a net cost on the two hardest families.

Table 1: Task success per system × query family ($n = 5$ each). Overall = unweighted mean of the four families.

System	Fact.	X-mod.	M.-hop	Conv.	Overall
V0	0.60	0.60	0.00	0.00	0.30
V1	1.00	1.00	0.20	0.00	0.55
V2	0.80	1.00	0.60	0.40	0.70
V3 (full)	1.00	1.00	0.40	0.20	0.65
V3-no-memory	1.00	0.80	0.60	0.00	0.60
V3-no-planner	1.00	0.80	0.60	0.60	0.75
V3-no-verifier	1.00	0.80	0.20	0.40	0.60

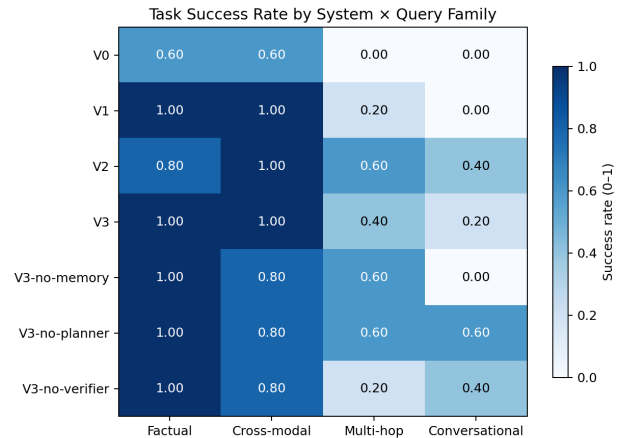


Figure 2: Task success heatmap across all seven configurations and four query families. Darker is better.

6.2 Where each component contributes positively

Retrieval is decisive. Adding a single text index (V0→V1) raises faithfulness from 0.00 to 1.00 on every family and lifts factual+cross-modal task success from 0.6 to 1.0. V0’s answers on multi-hop and conversational queries are 0% successful: without retrieval the LLM

cannot select among private recipes and either hallucinates (QF1.5) or politely defers (most of QF3, QF4).

Multimodal fusion helps multi-hop. V1→V2 lifts multi-hop from 0.20 to 0.60 (e.g. V2 newly passes QF3.1, QF3.2, QF3.5) by broadening retrieval through the image index — the image index sometimes surfaces candidates the text index misses for ingredient-combination queries.

Memory is essential for conversational. V3-no-memory scores 0/5 on conversational, identical to V0 and V1 on that family. The allergen-accumulation case (QF4.1) and the preference-conflict case (QF4.4) both depend on facts surfaced in turn 1 being applied in turn 2; without persistent memory there is no mechanism for this.

Verifier improves cross-modal grounding. V3 attains a visual grounding score of 0.93 (cross-modal queries only), versus 0.82 for V2 and 0.85 for V1: the verifier filters out candidates whose *visual attributes* do not match the query’s colour or presentation descriptors, raising the share of grounded descriptors in the final answer.

6.3 When planning hurts: a negative marginal benefit

Figure 3 isolates each V3 ablation against the full agent, making the planner regression visible by query family.

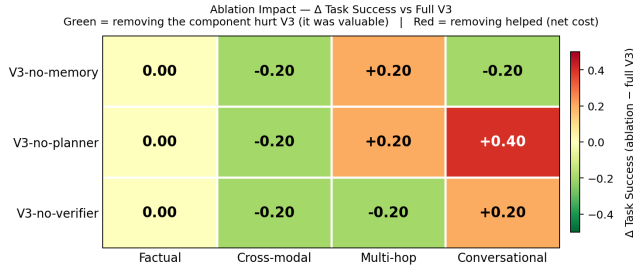


Figure 3: Per-family task-success delta of each ablation versus full V3. Negative bars mean removing the component hurt V3; positive bars mean removing it helped. The planner ablation is positive on multi-hop and conversational — structured planning was a net cost on these families.

The most surprising result is that the NO-PLANNER ablation *improved* task success on conversational (+0.40) and multi-hop (+0.20). On four V3 failures we manually inspected (QF3.1, QF3.3, QF4.4, QF4.5), the planner produced either an empty plan or a plan whose filter step over-constrained candidates from memory-derived facts; the verifier then removed the remaining candidates and the generator was forced to refuse (“I don’t have a matching Cantonese dish in my knowledge base”) even though the matching recipe was in the KB.

The reactive (bind_tools) path of V3-no-planner avoids this trap by retrieving aggressively across multiple tool turns. Its cost is correspondingly higher: an average of 15,955 tokens and 9.6 tool calls per query (Table 3) — ~3.6× the tokens of full V3 and over 16× those of V1. It is a Pareto-relevant configuration but not a Pareto winner.

6.4 Failure mode breakdown

Table 2: Failure-mode counts per system ($N = 20$ queries). Each failed record is classified into exactly one of five categories.

System	Retr.	Q-grnd.	Grnd.	Reas.	Cite	Total
V0	10	1	2	1	0	14
V1	2	2	1	4	0	9
V2	2	1	1	2	0	6
V3 (full)	3	0	1	1	2	7
V3-no-memory	4	0	3	1	0	8
V3-no-planner	0	0	1	2	2	5
V3-no-verifier	4	0	0	2	2	8

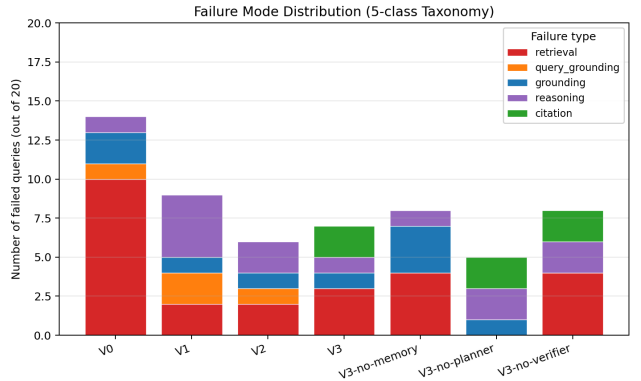


Figure 4: Failure taxonomy across systems. The stacked bars show how failures shift from missing retrieval in V0 toward reasoning, grounding, and citation errors once retrieval and agent components are introduced.

Table 2 and Figure 4 show the 5-class failure distribution. Three patterns stand out: (i) V0 is dominated by *retrieval* failures (10/14) — it cannot retrieve, so it hallucinates or defers; (ii) V1 and V2 shift toward *reasoning* failures, indicating that retrieval works but multi-hop aggregation breaks down (e.g. V1 on QF3.1 mentions only the winner without listing alternatives, failing *mentions-all-recipes*); (iii) V3’s failures concentrate in *retrieval* (empty candidates from over-constrained plans) and *citation* (recommends a recipe outside the constraint set in multi-turn cases). V3-no-planner has the fewest failures overall (5) and zero *retrieval* failures, again pointing at the structured plan as the brittle component.

6.5 Cost-quality trade-off

Table 3: Efficiency averages across all 20 queries.

System	Lat. (s)	Tokens	Tool calls
V0	4.5	390	0.0
V1	4.0	950	0.0
V2	5.9	1,490	0.0
V3 (full)	27.2	4,400	3.6
V3-no-memory	17.9	3,070	2.8
V3-no-planner	33.5	15,955	9.6
V3-no-verifier	19.5	3,970	3.0

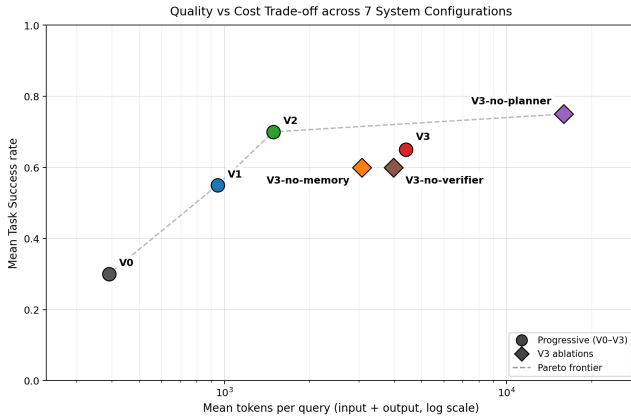


Figure 5: Cost-quality trade-off across configurations. Systems closer to the upper-left achieve higher task success with lower token cost; V3-no-planner improves quality but is the most expensive configuration.

Table 3 and Figure 5 show that V0 is $\sim 11\times$ cheaper than V3 and $\sim 41\times$ cheaper than V3-no-planner, while still attaining 0.6 on factual queries. The report-worthy implication is that simple systems are *operationally rational* for simple queries; the marginal benefit of agent complexity must be earned per query type.

7 Discussion: When does complexity pay off?

Synthesising the result tables, Table 4 summarises the configuration we would recommend per query family if cost were a binding constraint — which it normally is in production.

Table 4: Recommended configuration per query family, based on task success and token cost.

Family	Best system	Reason
Factual	V1	retrieval suffices; $\sim 10\times$ cheaper than V3
Cross-modal	V3 / V2	V3 best on grounding; V2 if cost-sensitive
Multi-hop	V2 / V3-no-mem.	both 0.60; V2 has half the latency
Conv.	V3-no-planner	only system > 0.4 ; pay for it

Two design implications follow. *First*, the often-cited mantra “retrieve-aggressively, filter-cautiously” is supported: V3 fails when its planner over-commits to a filter and the verifier prunes the last candidate. *Second*, the value of an agent depends on the *semantic ambiguity* of the query: factual look-ups need no agent; ambiguous multi-turn requests benefit from broad reactive retrieval. We see no single configuration that dominates the Pareto frontier.

8 Limitations and Future Work

Single LLM. All seven systems share DeepSeek V4 Flash for fair architectural comparison, but generalisation to GPT-4-class or Claude-class models is not verified. **KB scale.** 20 recipes is small; behaviour at 10^3+ recipes (where retrieval is the bottleneck rather than generation) may differ. **Planner prompt.** Our V3 planner was not heavily tuned; a more elaborated prompt with explicit “prefer broad over empty plan” rules likely closes much of the no-planner gap. We report the unmodified result because it preserves the integrity of the ablation. **Metric edge cases.** On QF3.2, V3 attains task success = 1 with Recall@3 = 0: the LLM’s prior knowledge happened to produce the right superlative without grounded retrieval. The same mechanism explains V0’s surprisingly high 0.60 on cross-modal queries — the LLM knows canonical visual properties (e.g. lasagna is layered) and gets credit under our programmatic constraints, which reward correct mention regardless of provenance. Both cases show task success and Recall@3 can decouple when surface heuristics align with gold answers.

9 Conclusion

Across 20 queries and 7 configurations, retrieval, multimodal fusion, memory, and verification each carry positive marginal value, but structured planning has *negative* marginal benefit on the two hardest query families. There is no single best agent; the best configuration depends on the query type and the cost budget. Rule-based programmatic evaluation made this conclusion auditable.

References

- [1] G. Ilharco *et al.*, “OpenCLIP,” Zenodo, 2021. https://github.com/mlfoundations/open_clip.
- [2] DeepSeek AI, “DeepSeek V4 Preview API Documentation,” 2026. <https://api-docs.deepseek.com/>.
- [3] TruEra, “The RAG Triad: Context Relevance, Groundedness, Answer Relevance,” TruLens Documentation, 2024.
- [4] LangChain, “LangGraph: Build stateful, multi-actor applications with LLMs,” 2024. <https://langchain-ai.github.io/langgraph/>.
- [5] Chroma, “Chroma: the AI-native open-source embedding database,” 2024. <https://www.trychroma.com/>.